

Smart Attendance System Using Face Recognition

Project Viva / Review — Complete Q&A; + All Diagrams

FastAPI · MongoDB · face_recognition · Vanilla JS / HTML / CSS

April 2026

#	Section
Q1	What is your project and why you chose it?
Q2	How to implement your project?
Q3	What is the Existing System?
Q4	Proposed Methods and why?
Q5	Data Sets / Sample Inputs and why?
Q6	Hardware & Software Requirements — enough?
Q7	Why the specific language — advantages?
Q8	Testing Methods — enough? Need for testing?
Q9	UML Models used and why?
Q10	Conclusion and Future Enhancements
FC-1A	Flowchart — Student Registration (Part A: Steps 1-10)
FC-1B	Flowchart — Student Registration (Part B: Steps 11-20)
FC-2	Flowchart — Login / Authentication
FC-3	Flowchart — Face-Based Attendance Marking
FC-4	Flowchart — Absent Notification Worker
UC	UML 1 — Use Case Diagram
CD	UML 2 — Class Diagram
SD-1	UML 3 — Sequence: Student Registration
SD-2	UML 4 — Sequence: Attendance Marking
SD-3	UML 5 — Sequence: Login
CP	UML 6 — Component Diagram
ER	UML 7 — ER Diagram

Q1. What is your project and why you chose it?

Project Overview

The **Smart Attendance System Using Face Recognition** is a full-stack web platform that automates student attendance using biometric facial recognition. It replaces slow, fraud-prone roll-call methods with an AI-powered camera-based system serving three roles: **Student**, **Teacher**, **Admin**, each with a dedicated web portal.

Key Features

- Student self-registration with webcam face capture and Gmail OTP verification.
- Teacher attendance marking via real-time camera — multiple student faces identified simultaneously.
- Manual attendance override, scoped to the teacher's own department.
- Student dashboard: monthly attendance calendar, percentage stats, and full history.
- Admin portal: user lifecycle management, audit logs, security review, report exports (Excel & PDF).
- Optional GPS geofencing — attendance may only be marked within a campus radius (Haversine formula).
- Automated daily absent notifications dispatched after a configurable IST cutoff (default 17:00).
- Duplicate biometric detection blocks registration fraud and writes a CRITICAL security log entry.
- Real-time WebSocket endpoint with polling fallback for live dashboard updates.

Why This Project

- Traditional methods (roll-call, sign sheets) are slow and trivially defeated by proxy signing.
- Face recognition is fast (multi-face scan in seconds), objective, and hard to spoof on a live webcam.
- Integrates advanced domains: computer vision, REST APIs, async programming, JWT auth, MongoDB — ideal as a capstone.
- Immediately deployable in any college with a webcam and internet connection.
- `face_recognition` library achieves 99.38% accuracy on LFW benchmark without a GPU.

Q2. How to implement your project?

Architecture

Monolithic full-stack: one FastAPI process serves both the REST API and static frontend via StaticFiles mounts. Uvicorn (ASGI server) handles all HTTP + WebSocket connections. MongoDB is the persistence layer via the Motor async driver.

Step 1 — Environment Setup

Python 3.11 venv. Install from requirements.txt: FastAPI, Uvicorn, Motor, face_recognition (dlib-bin on Windows), NumPy, OpenCV, ReportLab, Pandas, python-jose, passlib, python-dotenv.

Step 2 — Configure .env

Fill SECRET_KEY, MONGODB_URI, MAIL_*, CAMPUS_LAT/LON, FACE_TOLERANCE, LATE_HOUR, ABSENT_NOTICE_HOUR, and feature flags.

Step 3 — Database Setup

mongo.py creates unique indexes on users (id, email, roll_number), attendance (user_id+date), security_logs. Counters collection auto-increments all IDs.

Step 4 — Seed Accounts

create_admin.py + create_teachers.py populate the initial admin and teacher accounts per department.

Step 5 — Routes & Services

Routes (auth, teacher, student, attendance, admin) define HTTP endpoints by role. Services (face, attendance, user, email, report, security) contain all business logic.

Step 6 — Face Recognition Pipeline

Registration: Base64 JPEG → extract 128-D embedding → duplicate check → store in MongoDB. Attendance: classroom frame → multiple face encodings → match against dept students.

Step 7 — Auth (JWT + OTP)

OTP: 6-digit random → SHA-256+salt hash → email. Login: bcrypt verify → sign JWT (HS256, 60-min expiry). Protected endpoints validate JWT via get_current_user dependency.

Step 8 — Background Worker

asyncio task polls every 60 s. After cutoff, identifies absent students per dept, emails them, writes claim_doc to prevent double-send.

Step 9 — Frontend

Static HTML pages + shared JS (api.js auto-injects JWT Bearer) + CSS (glassmorphic design system).

Step 10 — Run

uvicorn main:app --host 127.0.0.1 --port 8000 --reload → http://127.0.0.1:8000/static/index.html

Q3. What is the Existing System of your project?

Traditional Attendance Methods

- **Manual Roll-Call:** Teacher calls names, marks paper register — 5–10 min/session, no fraud protection.
- **Paper Sign Sheets:** Students sign their names — trivially defeated by proxy signing.
- **Fingerprint Scanners:** Secure but expensive; students must queue; cannot scan many at once.
- **RFID / Smart Cards:** Cards can be lent, lost, or cloned for proxy attendance.
- **QR Code Check-In:** QR screen can be shared remotely, enabling off-campus proxy.
- **Standalone Desktop Software:** Excel/offline tools — no shared DB, no notifications, no analytics.

Limitations

- No biometric binding — all methods susceptible to proxy attendance.
- Manual entry wastes instructional class time.
- No automated absent notifications to students.
- Data silos — no simultaneous access for admin, teacher, and student.
- No real-time reporting, trend analytics, or audit trail.

Gap Filled by This Project

Every limitation above is directly addressed: face recognition eliminates proxy, camera-based marking takes seconds, automated emails alert absent students, a shared MongoDB database gives all roles real-time visibility, and every action is written to an immutable audit log.

Q4. What are the Proposed Methods and why you chose these methods?

Face Recognition — HOG + ResNet-34 Embeddings

dlib HOG locates faces; ResNet-34 maps each face to a 128-D float vector. Attendance granted when Euclidean distance $\leq$ FACE_TOLERANCE (0.5). 99.38% accuracy on LFW, CPU-only, one photo per student.

JWT Authentication (HS256)

Stateless signed tokens — no session table needed. sub=user email; configurable 60-min expiry. api.js injects Bearer token into every fetch automatically.

Email OTP Verification

6-digit random OTP, SHA-256+salt hashed (plain OTP never stored), expires in 10 min. Binds account to real Gmail ownership.

bcrypt Password Hashing

Adaptive, salted hash via passlib. Cost factor raised as hardware improves; rainbow-table attacks infeasible.

Haversine Geofencing (Optional)

Teacher GPS (navigator.geolocation) compared to campus coordinates via Haversine great-circle formula. Rejects marking if teacher is beyond MAX_DISTANCE_KM.

Async FastAPI + Motor

asyncio event loop — DB queries and email sends are awaited without blocking; many concurrent sessions on one thread.

Duplicate Biometric Detection

New face encoding vs. ALL active student encodings at registration. Match → deactivate existing; reject new registration; write CRITICAL security_log.

Background Absent Notification Worker

asyncio coroutine polls every 60 s. After cutoff: iterate depts, find absent students, email each, write claim_doc to prevent double-send.

Q5. What are the Data Sets / Sample Inputs and why chosen?

Live / Operational Inputs (Primary Dataset)

- **Student Face Images (Registration):** Single webcam photo per student. 128-D embedding stored — no offline dataset needed as ResNet-34 is pre-trained.
- **Teacher Classroom Frame:** Webcam frame with multiple faces. All encodings matched against the dept's stored embeddings.
- **GPS Coordinates:** navigator.geolocation provides lat/lon at each attendance event for geofencing and reporting.
- **IST Timestamps:** check_in stored as HH:MM:SS IST. Present vs. Late determined by configurable threshold (default 09:30).
- **Gmail Addresses + OTPs:** Gmail required; 6-digit OTPs bind account to real email ownership.

Seeded / Sample Data (Development & Demo)

- **seeded_users_export.csv:** 100+ pre-seeded student accounts for demo/testing.
- **create_teachers.py:** Seeds CS, Mechanical, Civil, etc. teacher accounts with known credentials.
- **create_admin.py:** Seeds the single system administrator account.
- **simulate_duplicate_face_incidents.py:** Inserts synthetic CRITICAL security log events for admin demo.

Why This Data Strategy

- Pre-trained ResNet-34 generalizes to any person — only one reference image per student needed.
- Storing 128-float embedding (~1 KB) instead of raw images keeps MongoDB compact.
- Limiting face comparison to teacher's dept reduces computation and false-positive risk.

Q6. Hardware & Software Requirements — are these enough or not and why?

Hardware Requirements

Component	Minimum	Recommended
CPU	i3 / Ryzen 3, 2 cores, 2 GHz	i5/i7 or Ryzen 5/7, 4+ cores
RAM	4 GB	8–16 GB
Storage	2 GB free	SSD with 10+ GB free
Webcam	720p (1.3 MP)	1080p for reliable face detection
Network	Wi-Fi / LAN	Stable LAN for multi-user deployment
GPU	Not required (CPU inference)	NVIDIA GPU for 1000+ student scale

Software Requirements

Category	Technology	Version / Notes
Language	Python	3.11 (required for dlib-bin Windows wheel)
Web Framework	FastAPI + Uvicorn	Latest stable (with websockets)
Database	MongoDB + Motor	6.x Community or Atlas
Face Recognition	face_recognition + dlib	1.3.0 / dlib-bin 19.24.6
Numerical / Image	NumPy, OpenCV, Pillow	1.24.3 / 4.8.1 / 12.0.0
Auth / Security	python-jose, passlib[bcrypt]	JWT HS256 + bcrypt
Reports	Pandas, openpyxl, ReportLab	Latest stable
Email	smtplib (stdlib)	Gmail SMTP + App Password
Frontend	HTML5, CSS3, Vanilla JS	No build step required

Are These Requirements Enough?

Yes — for departmental / single-campus deployment. CPU-only face recognition handles 20–40 students/dept in 1–3 s. MongoDB + Motor handles hundreds of concurrent sessions. FastAPI asyncio serves dozens of browser connections on one thread. **Scale-up path:** GPU inference, MongoDB replica set, Nginx + multiple workers, SendGrid/SES.

Q7. Why the specific language was chosen and its advantages?

Backend: Python 3.11

- **CV/ML Ecosystem:** face_recognition, dlib, NumPy, OpenCV are Python-native — no equivalent one-line embedding API elsewhere.
- **FastAPI:** Highest-performance Python web framework; auto-generates OpenAPI docs; uses type hints for zero-boilerplate validation.
- **Rapid Development:** Concise syntax + rich stdlib cuts boilerplate by 50–70% vs. Java.
- **Async-First (asyncio):** Non-blocking DB queries, email sends, and background worker — all on one thread.
- **Motor + pymongo:** Best-in-class async MongoDB drivers are Python-first.
- **Platform Portability:** Same codebase on Windows (dlib-bin wheel) and Linux without changes.
- **Readability:** Enforced indentation + 'plain English' style — easy to maintain.

Frontend: HTML5 + CSS3 + Vanilla JavaScript

- **Zero Build Step:** No npm, Webpack, or bundler — immediate development loop.
- **Full Design Control:** Vanilla CSS with custom properties: glassmorphism, animated backgrounds, gradient buttons.
- **Browser Native APIs:** WebRTC (camera), navigator.geolocation (GPS), Fetch API — used directly.
- **No Framework Lock-In:** Plain HTML works in every browser without Node.js tooling.
- **Lightweight:** No JS framework download — faster load on slow networks.

Q8. Testing Methods applied — are they enough? What is the need of testing?

Need for Testing

This system handles sensitive academic data (attendance records affecting grades), biometric information, JWT sessions, SMTP delivery, and security logging. Defects can cause data corruption, privacy breaches, or denial of service — making rigorous testing essential.

1. Module / Unit Smoke Tests

test_import.py: all backend modules import without errors. test_register.py: real POST /register inspects JSON response.

2. Integration Testing (FastAPI Swagger /docs)

Every endpoint tested in sequence: register → OTP verify → login → mark attendance → export report. Validates full request/response lifecycle including DI, middleware, serialization.

3. Database State Verification (check_db.py)

Directly queries MongoDB after migrations/seeds — prints security logs, config docs, counter values.

4. Boundary & Input Validation

Pydantic models enforce: email=EmailStr, password ≥ 6 chars, OTP exactly 6 digits, Base64 image non-empty. FastAPI returns HTTP 422 for violations.

5. Security / Negative Testing

Duplicate face: simulate_duplicate_face_incidents.py verifies CRITICAL log and account deactivation. Token expiry: confirms HTTP 401. Role guard: student JWT → /admin → HTTP 403.

6. Email Delivery Testing

Real Gmail SMTP tested live. Console-log fallback verified with MAIL_ENABLED=false. OTP 10-min expiry verified.

7. Geofencing Edge-Case Testing

Haversine utility tested with coordinates just inside and just outside the allowed radius.

8. Face Recognition Accuracy Tuning

Multiple registrations under varying lighting/angles. FACE_TOLERANCE tuned 0.6 → 0.5 to reduce false positives.

Are These Methods Enough?

For academic/pilot deployment: **Yes**. For production: add pytest + mongomock, CI/CD (GitHub Actions), Locust load testing, and OWASP ZAP pen-testing.

Q9. What type of UML models are used in your project and why?

1. Use Case Diagram

Actors: Student, Teacher, Admin, Email Service, MongoDB. Use cases: Register, Login, Mark Attendance (Face/Manual), View Dashboard, Export Reports, Manage Users, Configure Settings, Review Security Logs, Verify OTP, Receive Absent Email. WHY: Captures all actor-system interactions for requirements phase.

2. Class Diagram

Key Pydantic schemas (UserCreate, UserOut, AttendanceOut, TeacherStats, AdminConfig) and service classes (FaceService, AttendanceService, UserService, EmailService, ReportService). WHY: Documents static data model and inter-class relationships.

3. Sequence Diagram — Registration

Browser → POST /register → extract_encoding → duplicate check → create_student_user → send_otp_email → POST /verify-otp → mark_verified → success email. WHY: Shows exact temporal order of async calls.

4. Sequence Diagram — Attendance

Browser (Teacher) → POST /teacher/mark-attendance → geofence check → extract_multiple_encodings → find_matches → insert records → response. WHY: Maps the multi-service call chain clearly.

5. Sequence Diagram — Login

Browser → POST /login → get_user_by_email → verify_password → create_access_token → GET /users/me → role redirect. WHY: Clarifies the stateless JWT authentication flow.

6. Component Diagram

Components: FastAPI App, MongoDB, Browser, SMTP Gateway, face_recognition Library, WebSocket Manager. WHY: Shows deployment topology and in-process vs. external services.

7. ER Diagram

Entities: users, attendance, security_logs, config, counters, absent_notification_claims. Relationships: users 1-to-many attendance (user_id FK), users 1-to-many security_logs. WHY: Documents the MongoDB data model and application-layer referential integrity.

Q10. Conclusion and Future Enhancement of the project?

Conclusion

The **Smart Attendance System Using Face Recognition** demonstrates a production-grade, full-stack solution to a real-world institutional problem. By combining biometric face recognition, role-based web portals, email-driven OTP verification, optional geofenced attendance, and automated absence notifications in one FastAPI + MongoDB application, the project eliminates proxy attendance, reduces administrative overhead, and provides transparent, auditable records to all stakeholders.

Security-first design: bcrypt hashes, signed JWTs, duplicate-face detection with CRITICAL audit logging, and password re-confirmation for all privileged admin actions make the platform suitable for real institutional deployment. The lean Python + MongoDB + Vanilla JS stack is readable and maintainable.

Future Enhancements

- Liveness Detection (blink/head-pose challenge) to prevent photo/screen spoofing.
- Mobile App (Flutter/React Native) — native camera + GPS for teachers without laptops.
- GPU-Accelerated Inference (CUDA dlib / InsightFace) — reduce per-frame time from 2–3 s to <200 ms.
- Real-Time WebSocket Push — /ws endpoint and ConnectionManager already scaffolded; wire attendance_marked events.
- TOTP Multi-Factor Authentication — MFA_ENABLED flag exists; integrate pyotp for admin/teacher 2FA.
- Account Lockout Policy — LOCKOUT_ENABLED flag exists; implement 5-attempt lockout for 15 minutes.
- Subject-wise / Timetable-Integrated Attendance — per-subject percentage reports.
- Advanced Analytics Dashboard — weekly heatmaps, trend charts, early-warning for students below 75%.
- Docker + CI/CD Cloud Deployment — containerise, deploy to AWS/GCP/Azure via GitHub Actions + MongoDB Atlas.
- PWA / Offline Mode — service worker queues attendance actions when network is intermittent.

Module Assignment

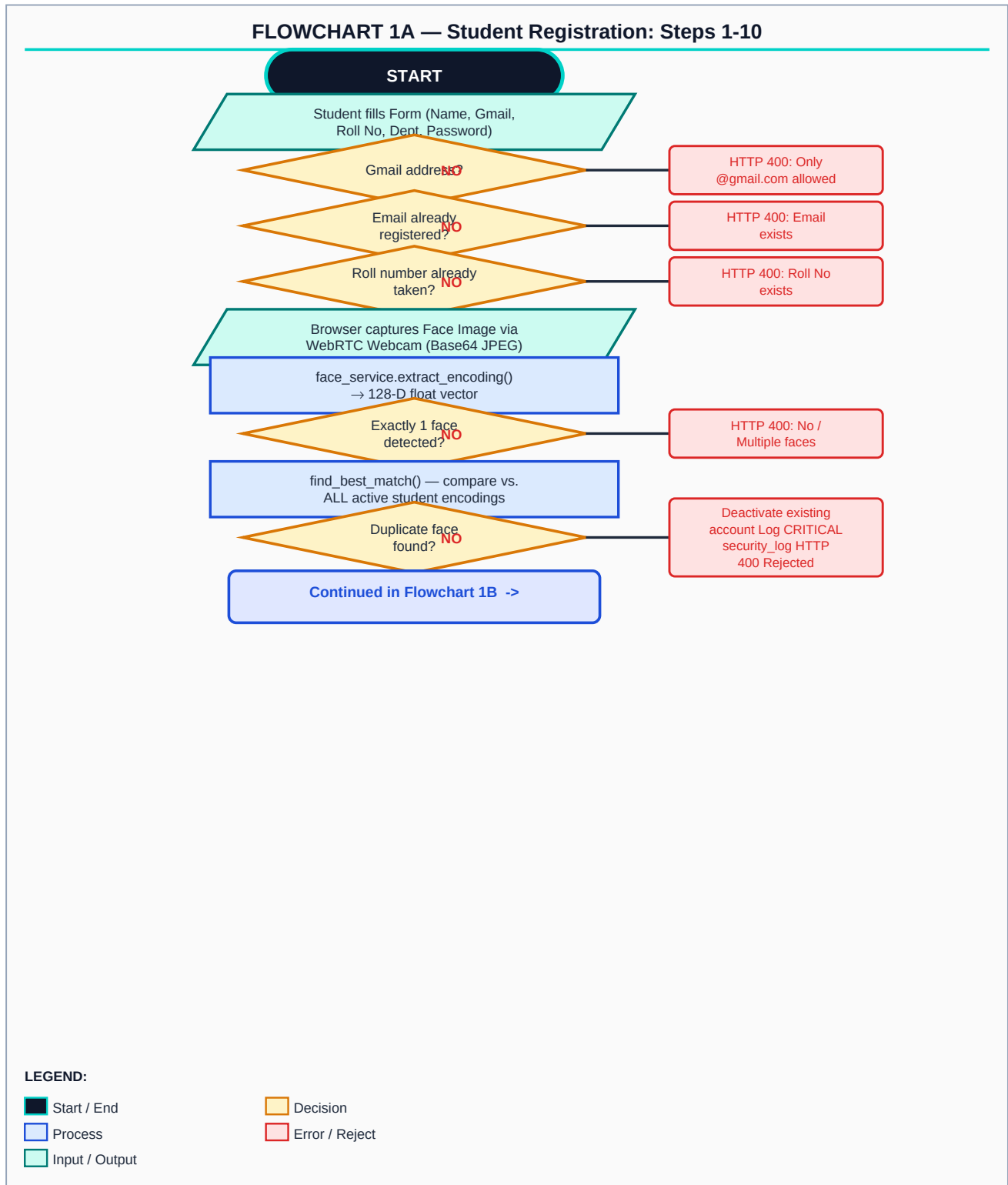
- Module 1 — Authentication & User Management (Registration, OTP, Login, Profile)
- Module 2 — Face Recognition Pipeline (Enrollment, Matching, Duplicate Detection)
- Module 3 — Attendance Marking & Finalization (Face + Manual, Geofencing, Cutoff Logic)
- Module 4 — Student Dashboard & Reporting (Calendar, Stats, History)
- Module 5 — Admin Portal (User Lifecycle, Config, Security Logs, Report Exports)
- Module 6 — Email & Notification Services (OTP, Absent Notices, Password Alerts)

DIAGRAMS & UML CHARTS

All Flowcharts and UML Diagrams — drawn to A4 page scale

FLOWCHART 1A — Student Registration Process (Steps 1–10: Validation & Face Check)

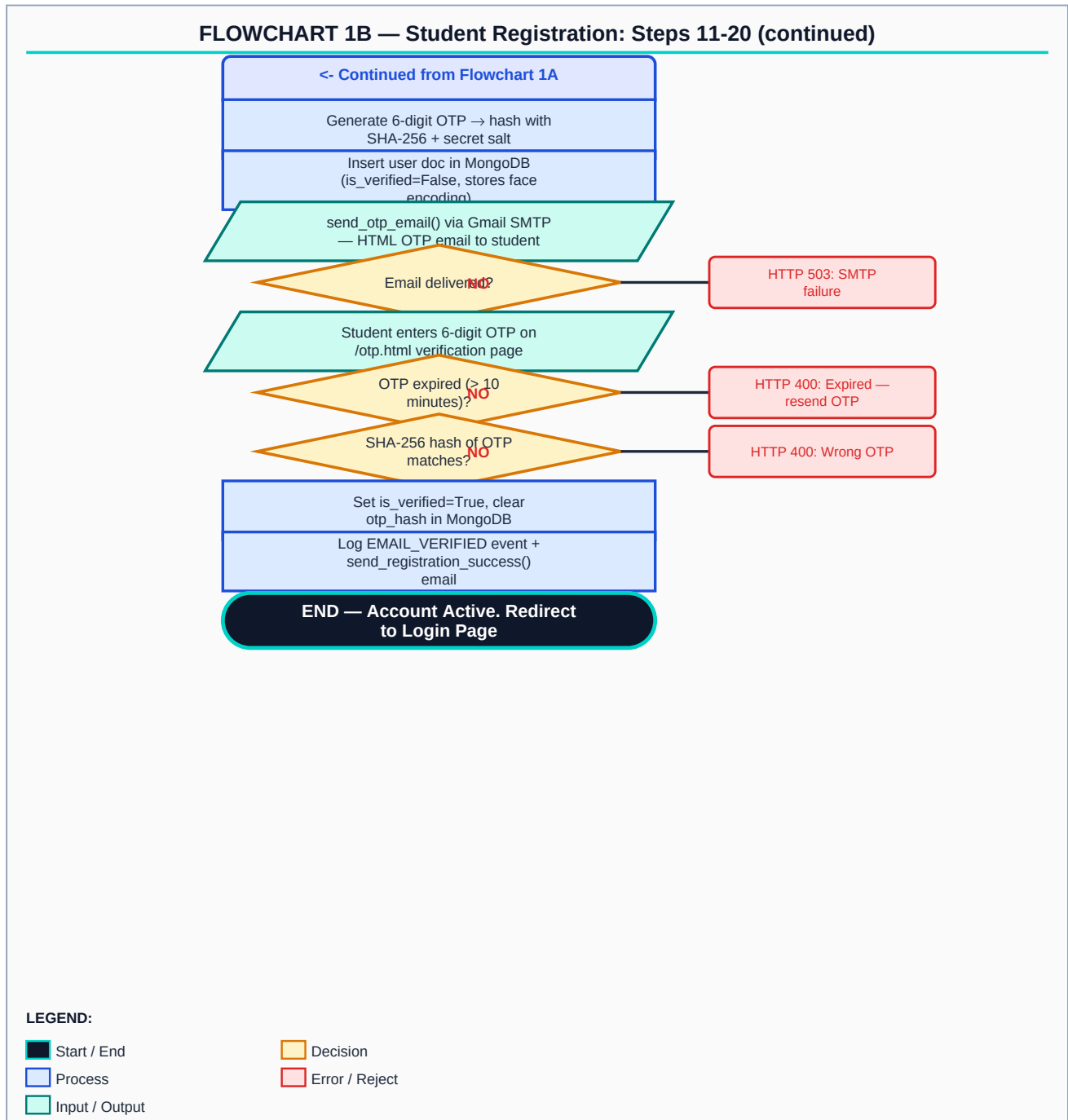
Input validation (Gmail, email uniqueness, roll uniqueness), webcam face capture, 128-D encoding extraction, and duplicate biometric detection. All error branches shown.



FC-1A: Registration Steps 1–10 — form validation, face capture, and duplication check

FLOWCHART 1B — Student Registration Process (Steps 11–20: OTP & Verification)

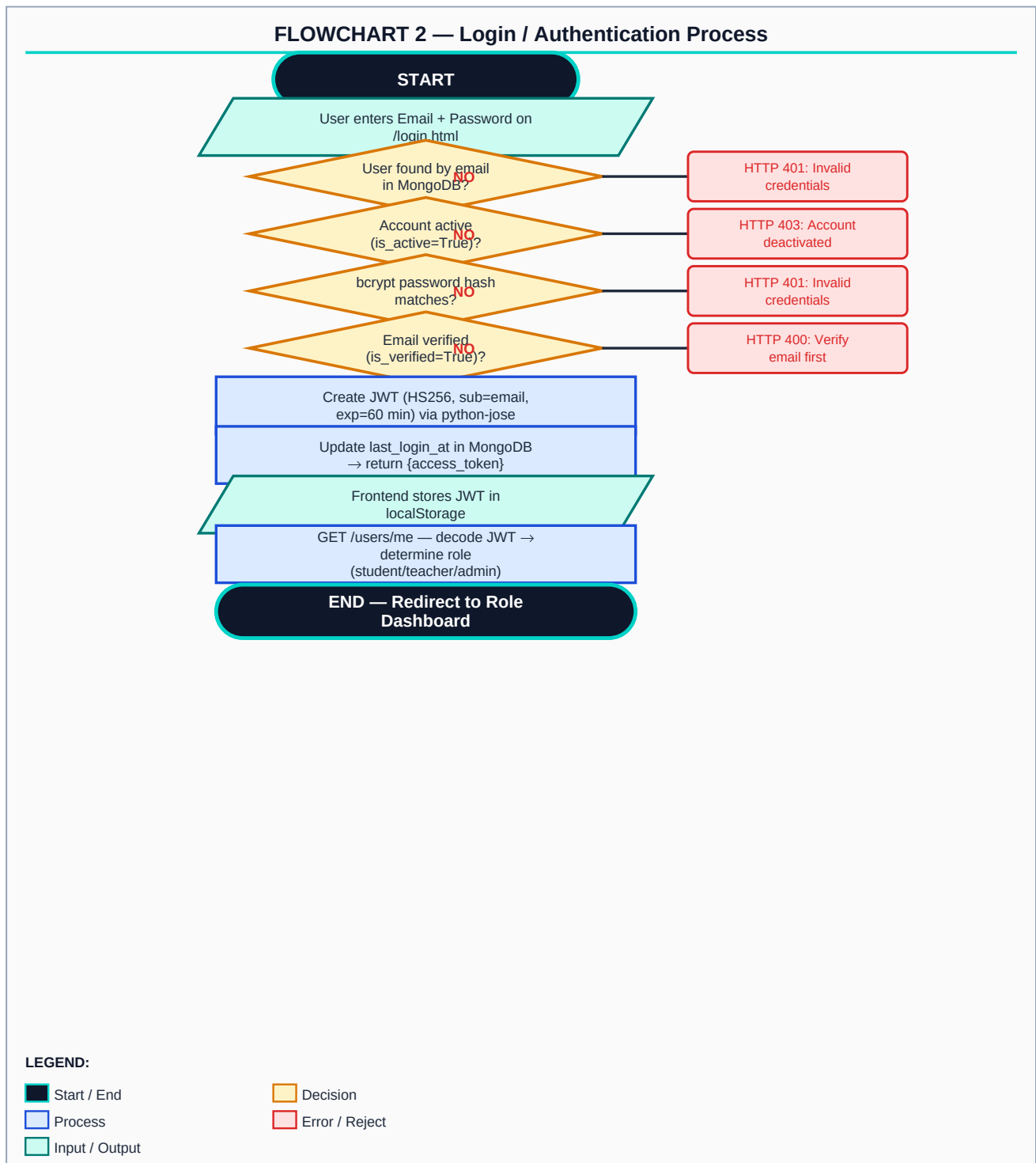
OTP generation, MongoDB user insertion, SMTP email dispatch, OTP entry, expiry check, hash verification, account activation, and success email dispatch.



FC-1B: Registration Steps 11–20 — OTP flow and account activation

FLOWCHART 2 — Login / Authentication Process

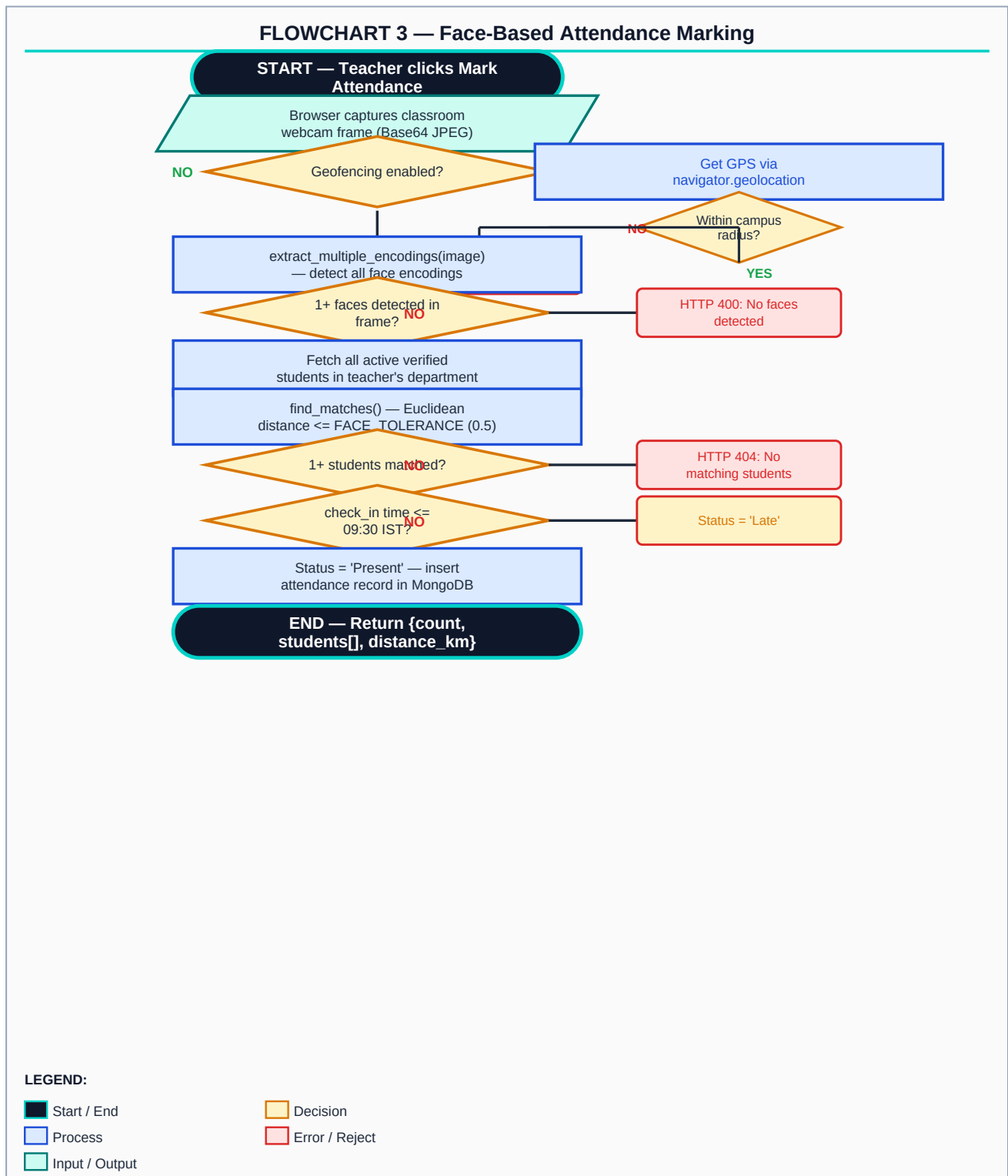
Email lookup, account status validation, bcrypt password check, email verification guard, JWT token creation, last_login_at update, and role-based dashboard redirection.



FC-2: Login and authentication flow with all validation and error branches

FLOWCHART 3 — Face-Based Attendance Marking

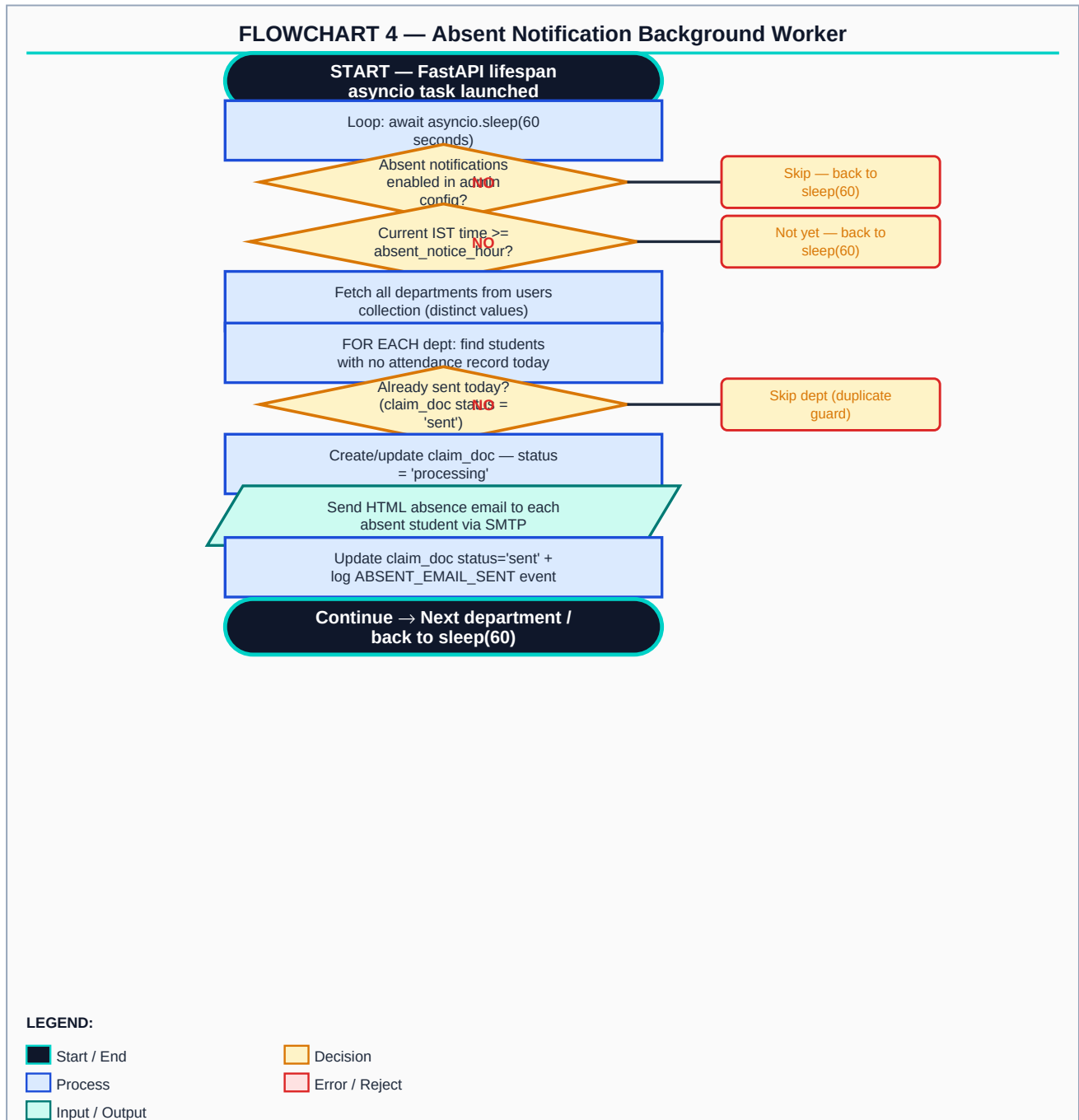
Teacher-initiated flow: webcam capture, optional geofencing, multi-face extraction, department student matching, and Present/Late status determination by IST check-in time.



FC-3: Face-based attendance marking with optional geofencing and multi-face match logic

FLOWCHART 4 — Absent Notification Background Worker

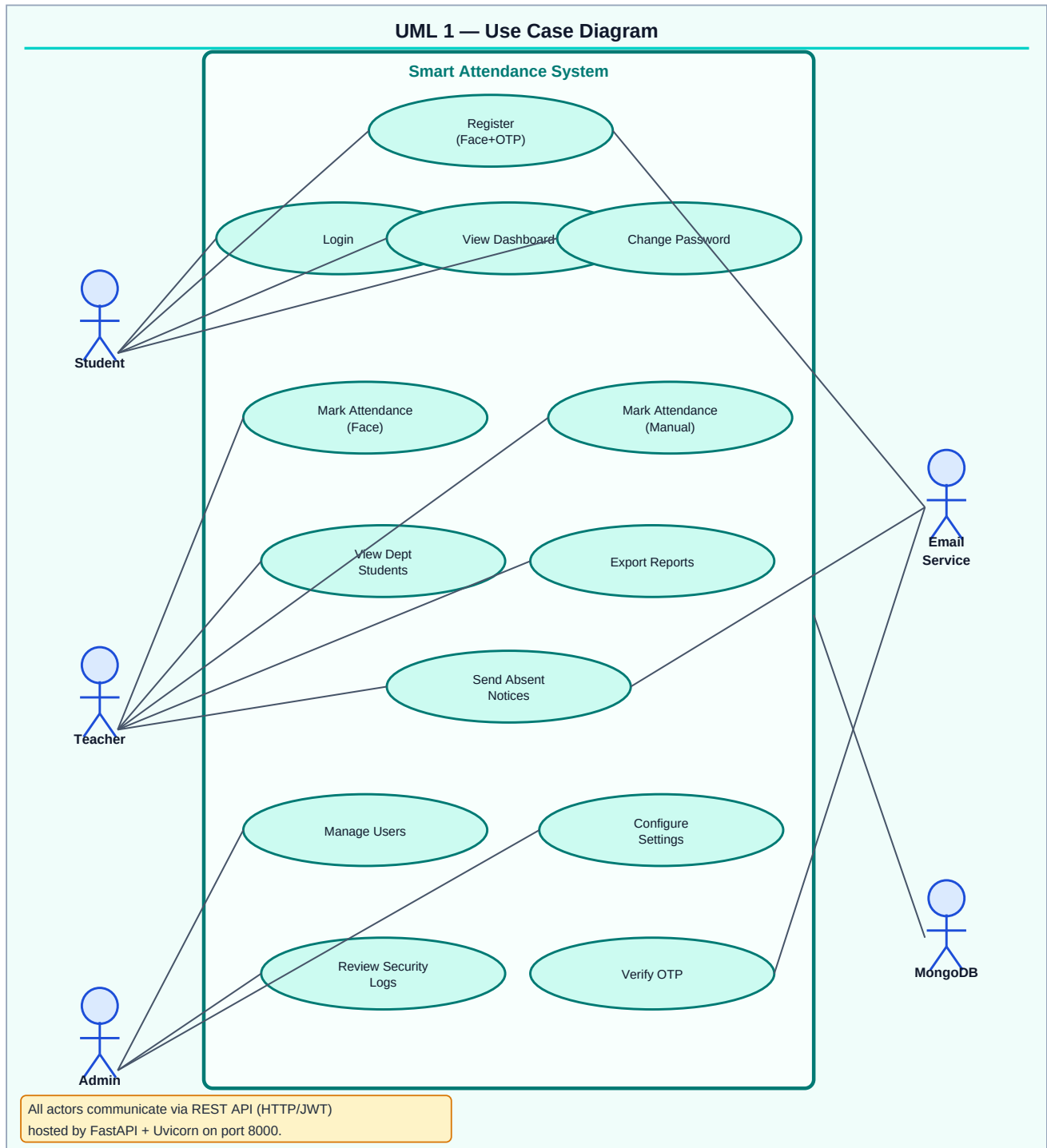
Long-running asyncio task that polls every 60 seconds, checks the IST cutoff, iterates departments, finds absent students, sends HTML emails, and prevents duplicate sends via claim documents.



FC-4: Background asyncio absent notification worker with duplicate-send prevention

UML DIAGRAM 1 — Use Case Diagram

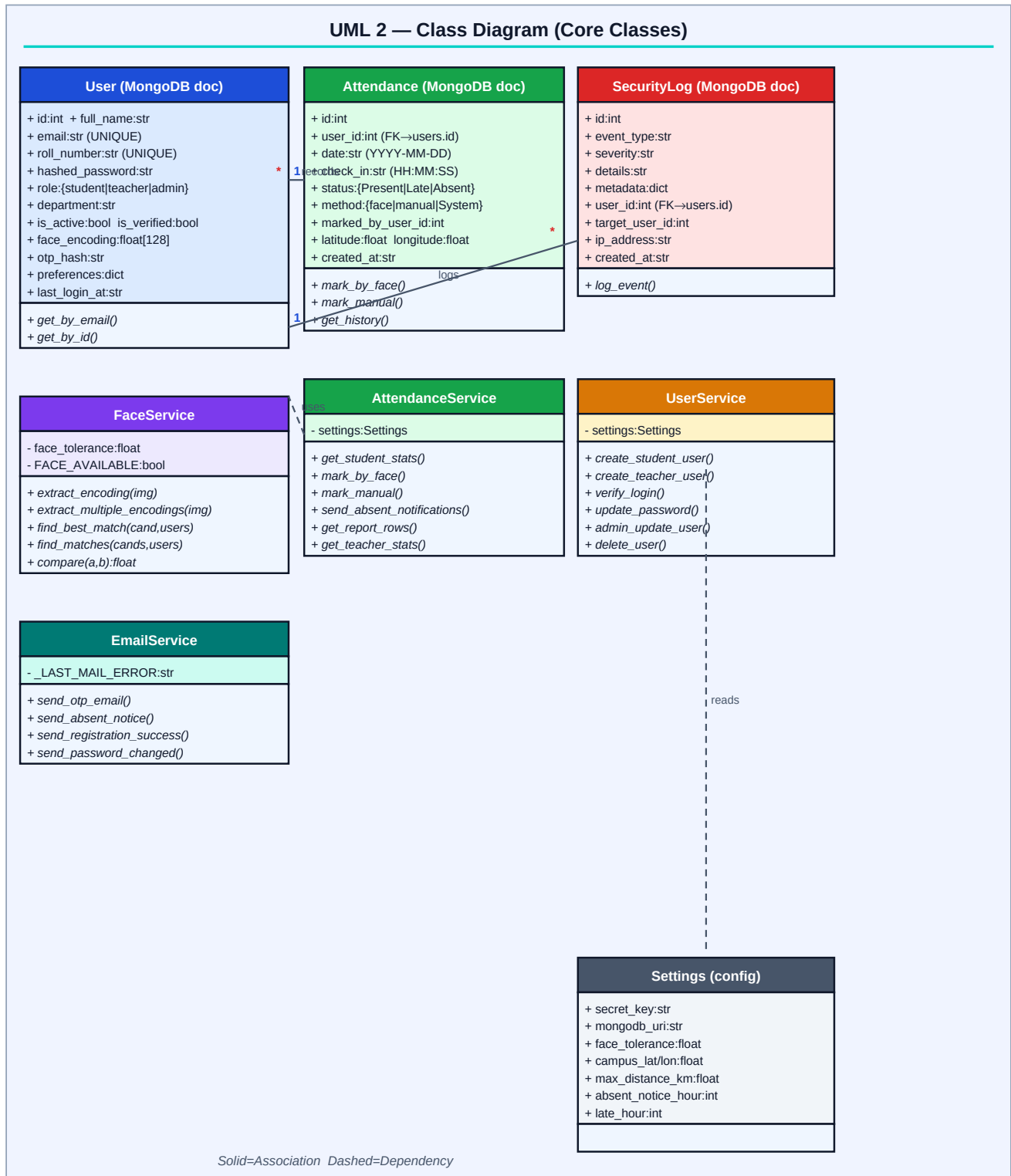
All actors (Student, Teacher, Admin, Email Service, MongoDB) and their use cases within the system boundary of the Smart Attendance System.



UML 1: Use Case Diagram — actors and their interactions with the system

UML DIAGRAM 2 — Class Diagram (Core Classes)

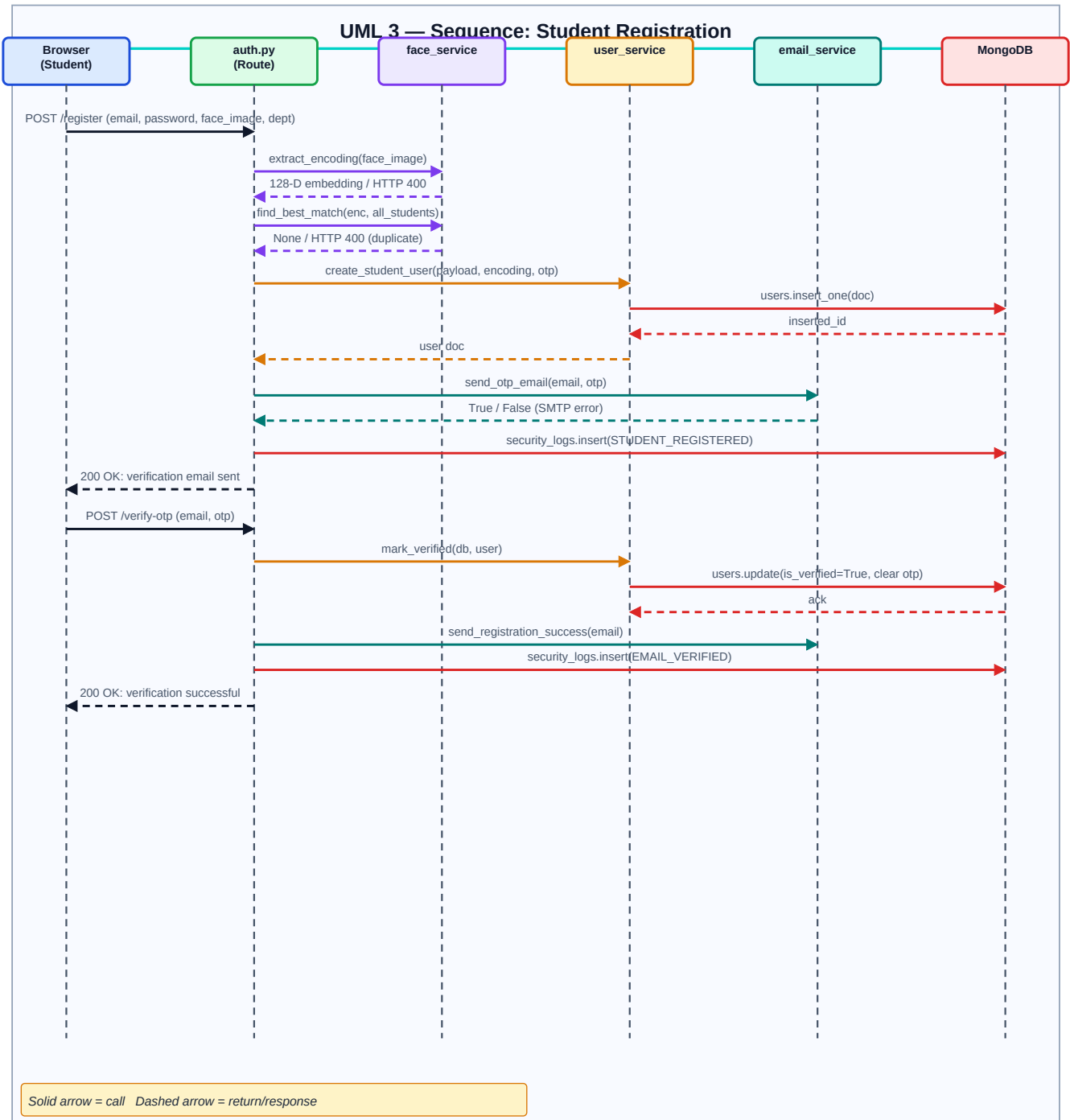
MongoDB document classes (User, Attendance, SecurityLog), service classes (FaceService, AttendanceService, UserService, EmailService), and Settings configuration dataclass, with attributes, methods, and inter-class relationships.



UML 2: Class Diagram — core data models and service classes with associations

UML DIAGRAM 3 — Sequence Diagram: Student Registration

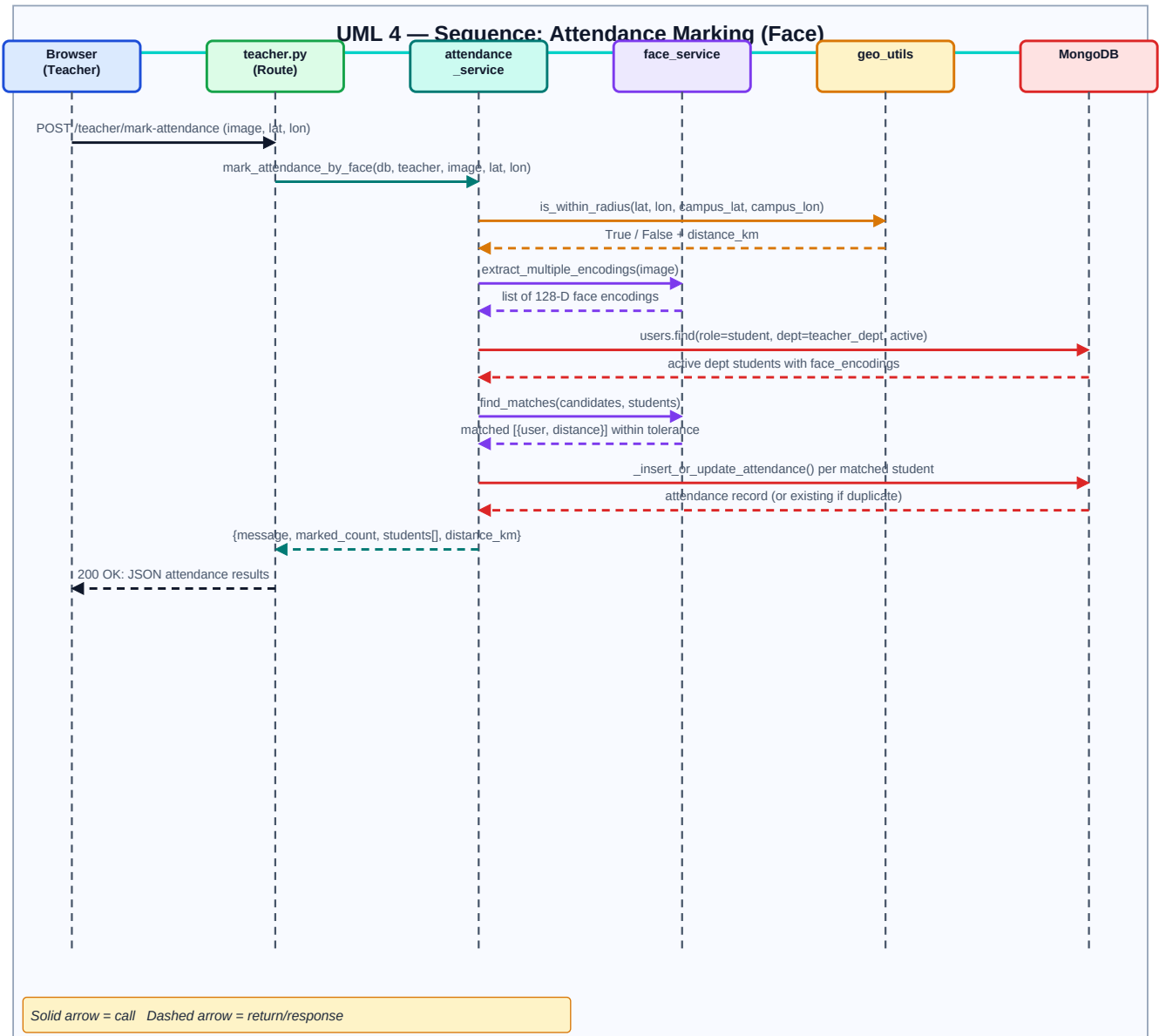
Complete temporal sequence of method calls between Browser, auth.py route, face_service, user_service, email_service, and MongoDB during student registration and OTP verification.



UML 3: Sequence Diagram — Student registration with all participants and messages

UML DIAGRAM 4 — Sequence Diagram: Attendance Marking (Face)

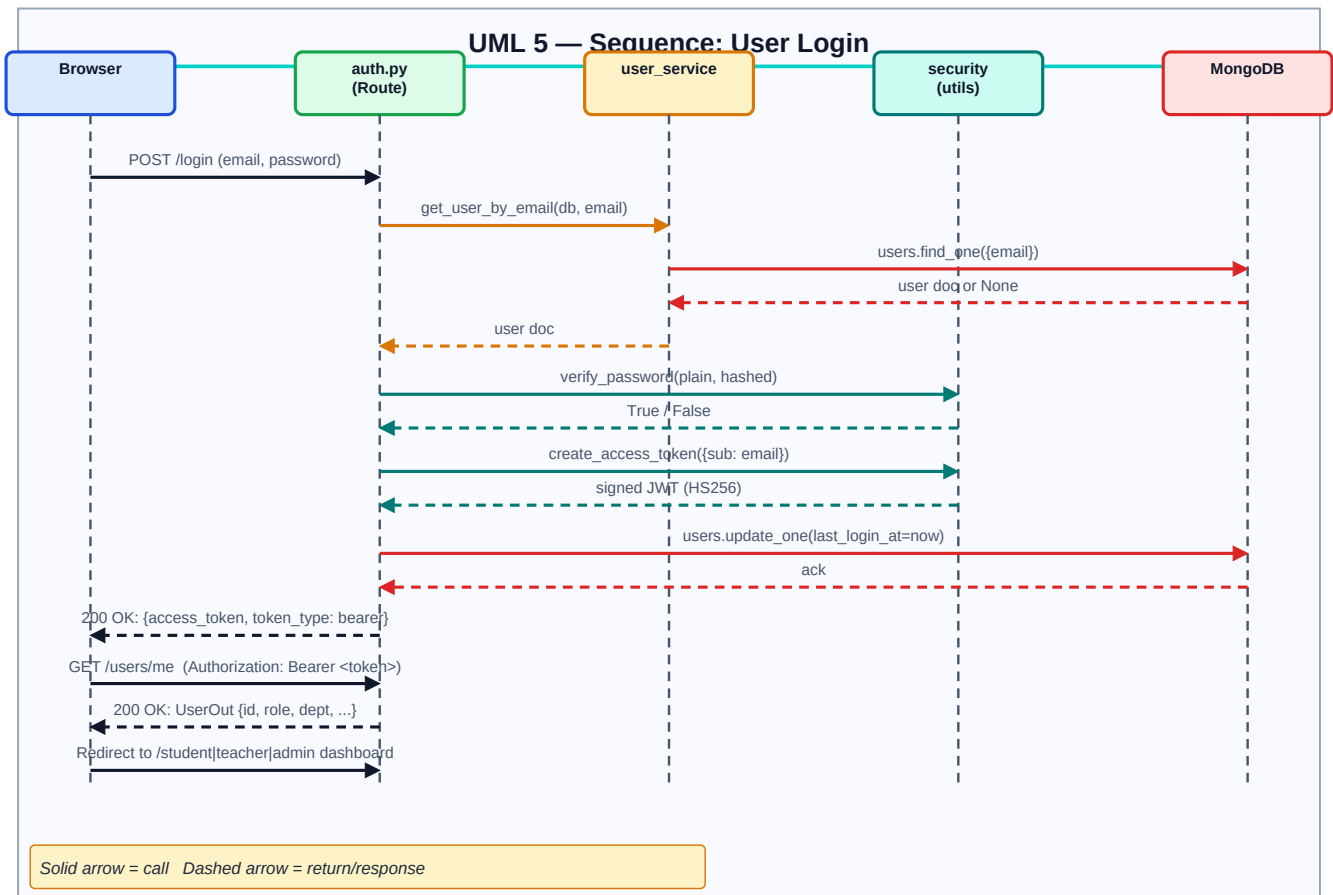
Sequence of calls between Browser (Teacher), teacher.py route, attendance_service, face_service, geo_utils, and MongoDB during a face-based attendance marking event.



UML 4: Sequence Diagram — Teacher face-based attendance marking process

UML DIAGRAM 5 — Sequence Diagram: User Login

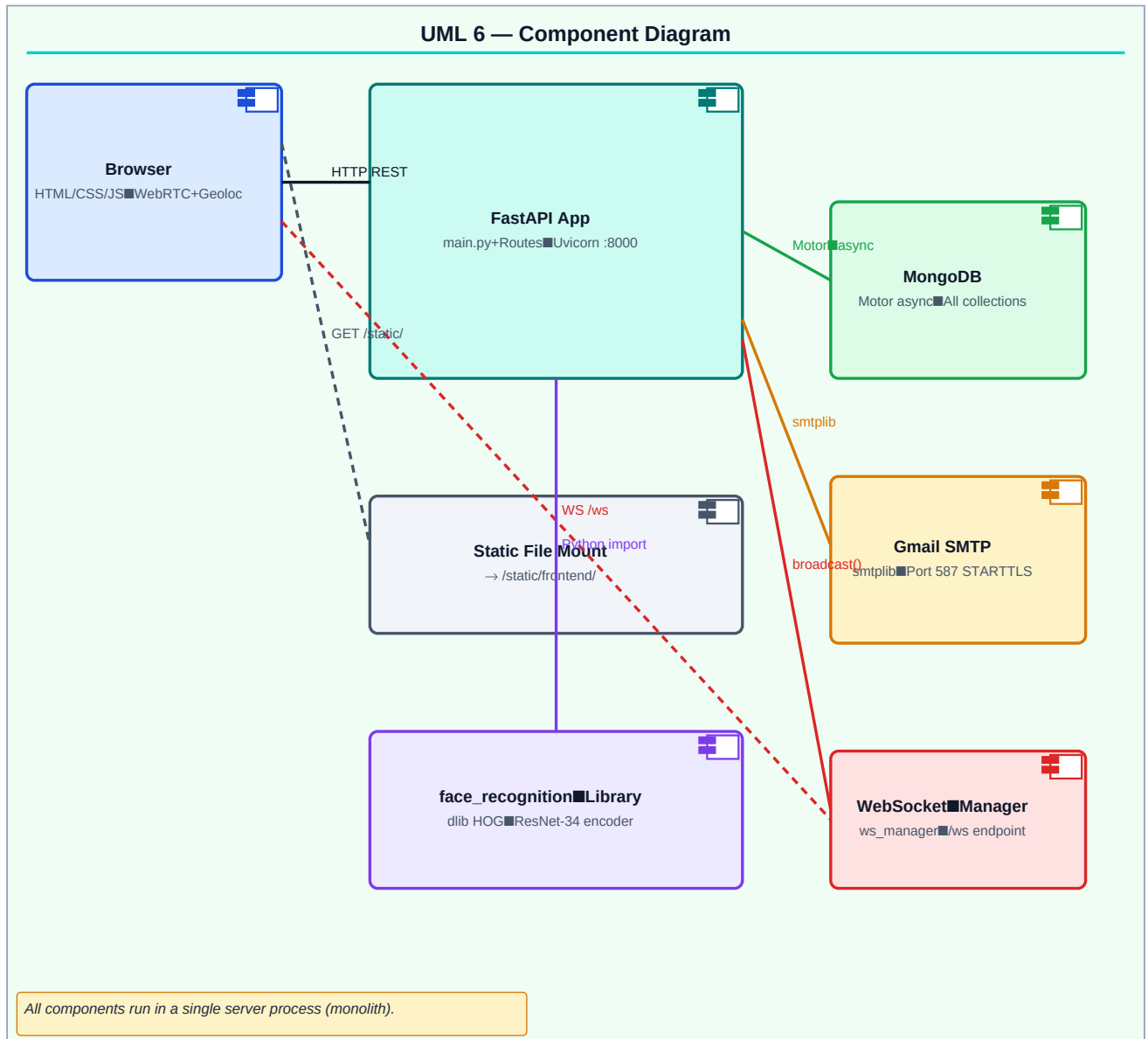
Sequence during login: email lookup, bcrypt verification, JWT token creation, last_login_at update, and the GET /users/me call that determines role-based redirect.



UML 5: Sequence Diagram — User login and JWT authentication flow

UML DIAGRAM 6 — Component Diagram

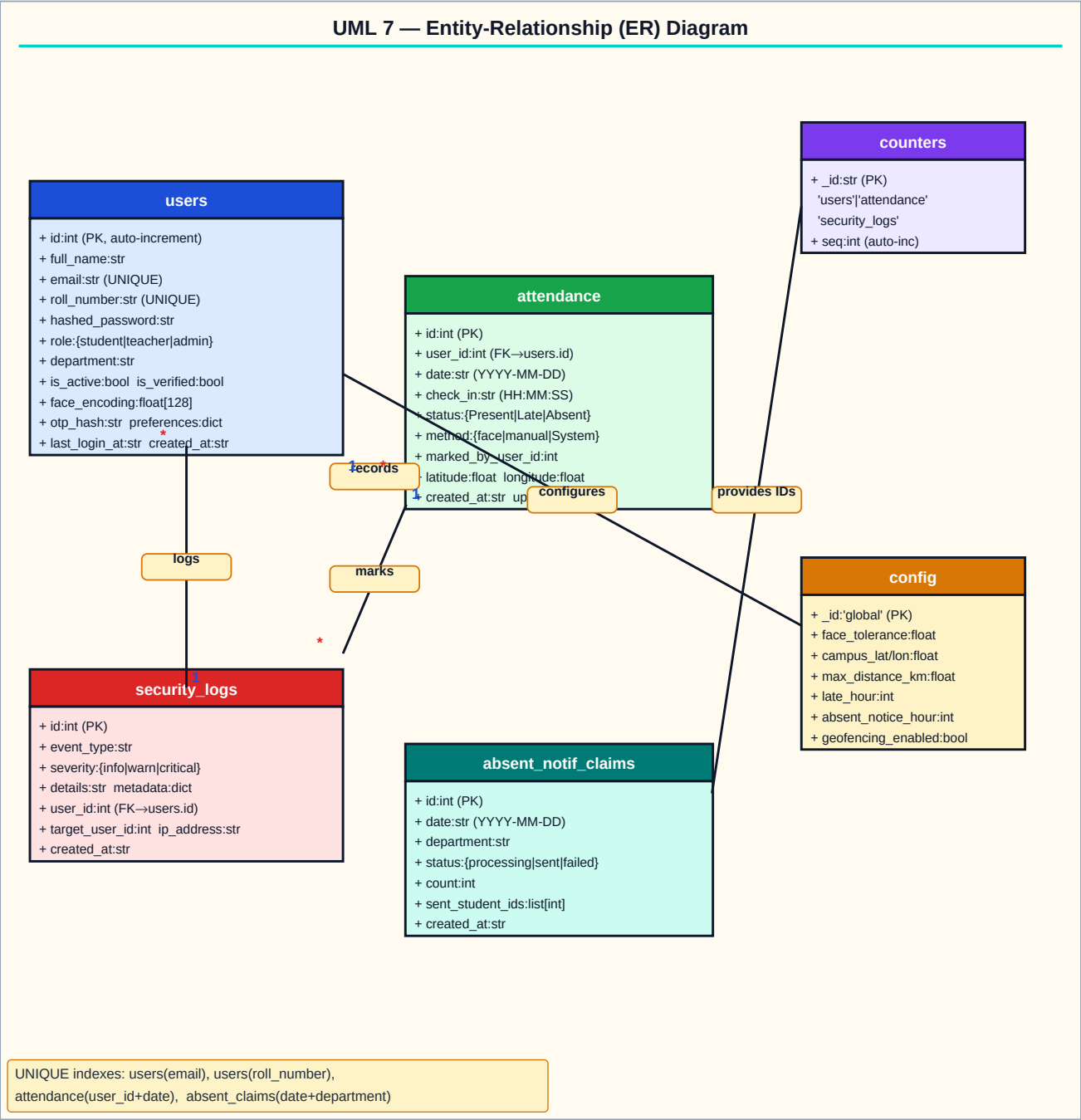
Major components — Browser (Frontend), FastAPI Application, Static File Mount, MongoDB, Gmail SMTP, face_recognition Library, WebSocket Manager — with interfaces and protocols.



UML 6: Component Diagram — system components and their interfaces

UML DIAGRAM 7 — Entity-Relationship (ER) Diagram

All MongoDB collections as entities — users, attendance, security_logs, config, counters, absent_notification_claims — with all fields, primary keys, foreign key references, and cardinality relationships.



UML 7: ER Diagram — all MongoDB collections with fields and relationships